

PROJECT 31A — TEMPORAL WORKFLOW & ORCHESTRATION BIBLE (DOCUMENT 4)

Version 1.0.0

Classification: CONFIDENTIAL — FOR INTERNAL DEVELOPMENT AND DUE DILIGENCE ONLY

1. TEMPORAL ORCHESTRATION ARCHITECTURE OVERVIEW

Project 31A utilizes **Temporal.io** as its core distributed orchestration engine. Unlike simple message queues or schedulers (e.g. Celery), Temporal enables durable, stateful execution of complex business logic.

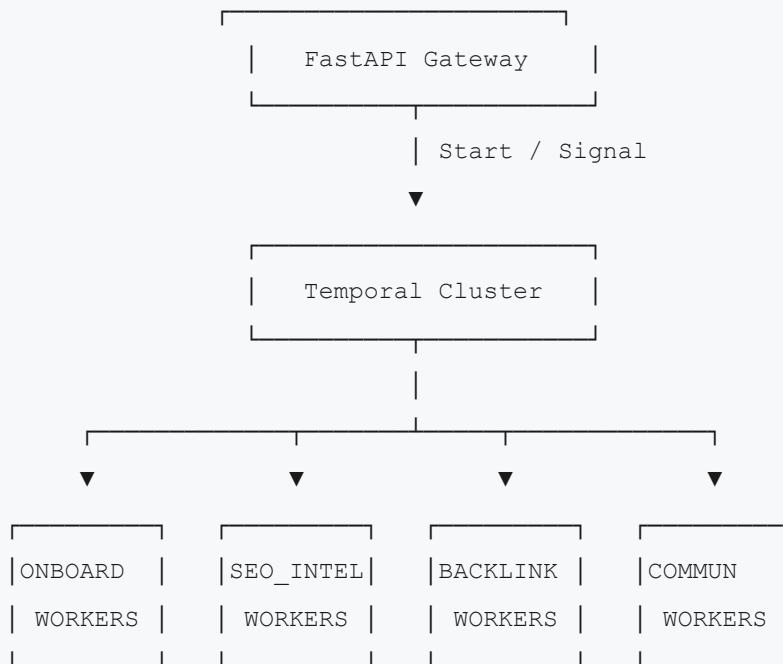
Workflows are written in standard Python, but their execution state is automatically checkpointed by the Temporal server. In the event of a worker crash, network partition, or database outage, the running workflow automatically resumes on a different worker at the exact point of interruption without losing state or data consistency.

1.1 Core Execution Mechanics

- **Durable Execution:** Workflows use `workflow.wait_condition` and async logic to suspend execution without blocking threads.
 - **Deterministic Replay:** Workflows must be strictly deterministic. Replay is used to reconstruct workflow state. Direct network calls, filesystem operations, and time lookups are forbidden inside workflows and must be executed in **Activities**.
 - **Signals:** External entities (users, inbound email webhooks) send information to running workflows using `workflow.signal`.
 - **Queries:** Running states (e.g., current prospect queue status) can be read using `workflow.query`.
 - **Child Workflows:** Complex processes (like sending emails per prospect domain) are spawned as child workflows via `workflow.execute_child_workflow` to decouple retry domains.
-

2. TASK QUEUE REGISTRY

Workflows and activities are routed to specific worker groups via dedicated **Task Queues**. The registry is declared in `backend/src/seo_platform/workflows/__init__.py` and registered by `backend/src/worker.py`.



2.1 ONBOARDING Queue

- **Workflows:** `OnboardingWorkflow`
- **Activities:** `validate_client_domain_activity`, `enrich_business_profile_activity`, `discover_competitors_activity`, `save_onboarding_results_activity`.
- **Worker pool sizing:** 5 concurrent processes.

2.2 SEO_INTELLIGENCE Queue

- **Workflows:** `KeywordResearchWorkflow`
- **Activities:** `expand_keywords_activity`, `enrich_keywords_activity`, `generate_keyword_embeddings_activity`, `cluster_keywords_activity`, `name_clusters_activity`.
- **Worker pool sizing:** 10 concurrent processes.

2.3 BACKLINK_ENGINE Queue

- **Workflows:** `BacklinkCampaignWorkflow`
- **Activities:** `discover_prospects_activity`, `fallback_prospects_activity`, `score_prospects_activity`, `discover_contacts_activity`, `create_approval_request_activity`, `update_campaign_status_activity`, `record_timeline_step_activity`, `raise_workflow_failure_alert_activity`.
- **Worker pool sizing:** 25 concurrent processes.

2.4 AI_ORCHESTRATION Queue

- **Workflows:** None (activity-only queue)
- **Activities:** `generate_outreach_emails_activity`, `generate_ai_summary_activity`.

- **Worker pool sizing:** 15 concurrent processes. Dedicated queue to handle long-running LLM model generation logic.

5.5 COMMUNICATION Queue

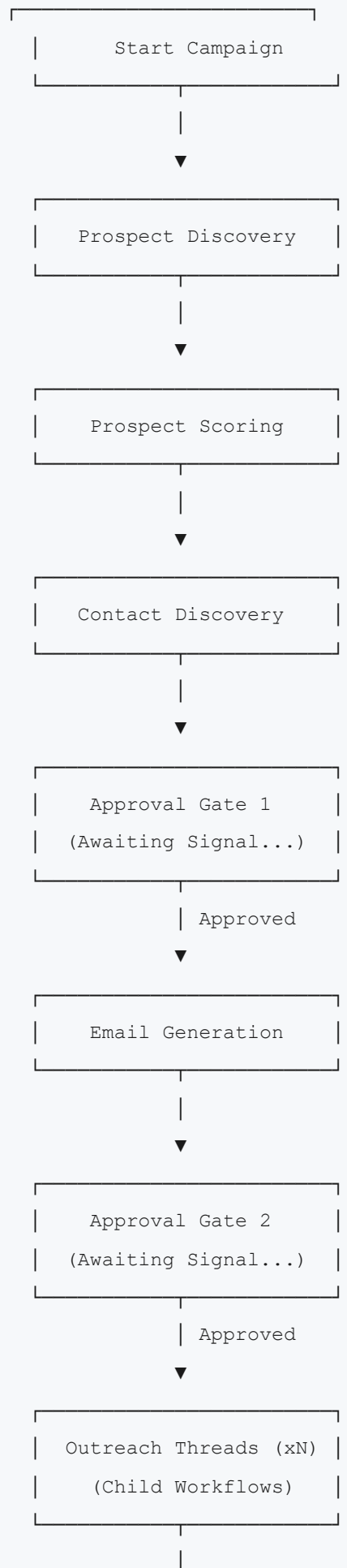
- **Workflows:** `OutreachThreadWorkflow`
- **Activities:** `send_outreach_batch_activity`, `send_single_email_activity`, `verify_email_delivery_activity`.
- **Worker pool sizing:** 20 concurrent processes.

5.6 REPORTING Queue

- **Workflows:** `ReportGenerationWorkflow`
- **Activities:** `gather_report_data_activity`, `persist_report_activity`.
- **Worker pool sizing:** 5 concurrent processes.

3. `BacklinkCampaignWorkflow` COMPLETE EXECUTION TRACE

The `BacklinkCampaignWorkflow` is the primary operations path. The complete execution sequence is defined in `backend/src/seo_platform/workflows/backlink_campaign.py`.



Monitoring / Complete

3.1 Step-by-Step Activities Execution Detail

Step 1: Initialize Campaign Timeline & Status

- **Action:** Spawn activity to set initial status to `prospecting`.
- **Activity:** `update_campaign_status_activity`
- **Arguments:** `(tenant_id, campaign_id, CampaignStatus.PROSPECTING)`
- **Task Queue:** `BACKLINK_ENGINE`
- **Timeout:** `StartToClose = 30 seconds`
- **Retry Policy:** `DATABASE` preset
- **Timeline log:** `record_timeline_step_activity(campaign_id, 'discovery', 'processing')`

Step 2: Discovered Prospects Acquisition

- **Action:** Query referring domains of competitors.
- **Activity:** `discover_prospects_activity`
- **Arguments:** `(tenant_id, campaign_id, competitor_domains)`
- **Task Queue:** `BACKLINK_ENGINE`
- **Timeout:** `StartToClose = 10 minutes`
- **Retry Policy:** `EXTERNAL_API` preset (5 attempts, max backoff 5 minutes)
- **Logic:**
 1. Iterates over competitors, calling `ahrefs_client.get_referring_domains(domain)`.
 2. If `AhrefsRateLimitError` is raised, execution is immediately failed-fast (non-retryable for this execution block).
 3. If Ahrefs fails for other reasons, it falls back to the configured unified SEO provider: `get_seo_provider().discover_backlink_prospects(domain, limit=20)`.
 4. If both fail, falls back to direct DOM scraper: `backlink_scraper.discover_link_intersect_prospects()`.
- **Output:** Returns `{prospects: List[dict], count: int}`.

Step 3: Zero-Prospect Fallback Check

- **Action:** If count is 0, execution branches.
- **Branch:**
 - **Activity:** `fallback_prospects_activity`
 - **Arguments:** `(tenant_id, campaign_id)`
 - **Task Queue:** `BACKLINK_ENGINE`

- **Timeout:** `StartToClose = 2 minutes`
- **Retry Policy:** `DATABASE` preset
- **Logic:** Queries DB for high-relevance prospects discovered during previous campaigns for the same tenant.
- **Fail-Loud Guard 1:** If count remains `0` after fallback, the workflow terminates immediately.
 - Updates campaign status to `failed_no_prospects`.
 - Sets timeline status to `failed`.
 - Raises alert.
 - *Constraint:* Platform **refuses** to proceed with synthetic or empty inputs.

Step 4: Prospect Scoring

- **Action:** Calculate metric compatibility.
- **Activity:** `score_prospects_activity`
- **Arguments:** `(tenant_id, campaign_id, prospects, min_da, max_spam, target_niche)`
- **Task Queue:** `BACKLINK_ENGINE`
- **Timeout:** `StartToClose = 15 minutes`
- **Retry Policy:** `EXTERNAL_API` preset
- **Logic:**
 1. For each prospect, gets domain authority from Ahrefs or unified SEO provider.
 2. Runs `backlink_relevance.analyze_prospect()` matching page text against client niche.
 3. Runs spam detector: `backlink_intelligence.detect_link_farm_and_spam()`.
 4. Combines results: `final_spam = max(analysis.spam_score, farm_analysis.spam_score)`.
 5. Multiplies composite score by `(1.0 - final_spam)` if spam > 0.3.
 6. Filters out prospects with composite score < 0.35 or `is_spam == True`.

Step 5: Contact Discovery

- **Action:** Find deliverable target emails.
- **Activity:** `discover_contacts_activity`
- **Arguments:** `(tenant_id, prospects)`
- **Task Queue:** `BACKLINK_ENGINE`
- **Timeout:** `StartToClose = 10 minutes`
- **Retry Policy:** `EXTERNAL_API` preset
- **Logic:**
 1. Passes domain list to Hunter API client.
 2. Performs Hunter domain search to retrieve up to 3 email candidates.
 3. For each candidate, calls `hunter_client.verify_email()`.
 4. Maps to internal state (deliverable/undeliverable/risky/unknown).
 5. Selects first `deliverable` email.
 6. Rejects prospects with no deliverable email (marks status `REJECTED`).

- **Fail-Loud Guard 2:** If the number of prospects with active emails is `0`, the campaign exits with status `failed_no_prospects`.

Step 6: Approval Gate 1 (Prospect Approval)

- **Action:** Create approval request and block.
- **Activity:** `create_approval_request_activity`
- **Arguments:** `(tenant_id, run_id, "prospect_approval", "medium", summary, context)`
- **Task Queue:** `BACKLINK_ENGINE`
- **Timeout:** `StartToClose = 2 minutes`
- **Retry Policy:** `TRANSIENT_IDEMPOTENT` preset
- **Logic:**
 1. Generates an idempotency key `approval:{run_id}:prospect_approval`.
 2. Writes an `ApprovalRequest` record to the DB.
 3. Updates campaign status to `awaiting_approval`.
 4. Calls `workflow.wait_condition(lambda: self._prospects_approved is not None)` which suspends execution.
 5. Replays check when external Signal `prospect_decision` is received.
 6. If signal payload is `rejected`, sets campaign status to `cancelled` and returns.

Step 7: Email Personalization Sequence Generation

- **Action:** Generate initial email and two follow-ups.
- **Activity:** `generate_outreach_emails_activity`
- **Arguments:** `(tenant_id, campaign_id, approved_prospects, campaign_type)`
- **Task Queue:** `AI_ORCHESTRATION`
- **Timeout:** `StartToClose = 30 minutes`
- **Retry Policy:** `LLM_INFERENCE` preset
- **Logic:**
 1. Calls `website_analyzer` to parse page metadata and paragraph highlights.
 2. Queries relationship store for past emails.
 3. Requests generation from `llm_gateway.complete(TaskType.SEO_ANALYSIS)`.
 4. Runs `OutreachEmailSchema` parser check on results: ensures no generic templates, validates semantic grounding ratio ≥ 0.2 .
 5. Writes `OutreachThread` database records.
- **Fail-Loud Guard 3:** If count of successfully generated emails is `0`, the workflow exits with `failed_no_emails_sent`.

Step 8: Approval Gate 2 (Outreach Content Approval)

- **Action:** Blocks until user approves the generated text templates.
- **Signal:** `outreach_decision`

- **Logic:** Suspends workflow using `wait_condition` exactly like Gate 1.

Step 9: Outreach Dispatches Initiation

- **Action:** Spawns child workflows.
- **Task Queue:** `COMMUNICATION`
- **Workflow:** Spawns an instance of `OutreachThreadWorkflow` per prospect.
- **Logic:** Uses child workflows to isolate retry policies, enabling individual threads to run, wait, retry, and manage replies without blocking other prospects.

Step 10: Campaign Monitoring and Termination

- **Action:** Tracks overall metrics.
- **Logic:** Loop iterates every 24 hours, querying DB for acquired links. Once `acquired_link_count > target_link_count` or workflow time context exceeds duration (e.g. 30 days), sets status to `complete`.

4. OutreachThreadWorkflow (CHILD WORKFLOW)

Spawned per prospect to manage the 3-stage outreach sequence.

4.1 Execution Sequence Flow

1. Stage 1: Dispatch Initial Email

- Checks `kill_switch_service.is_blocked('email_sending', tenant_id)`.
- Sends email via `send_single_email_activity` (resend/sendgrid/mailgun).
- Suspends execution: `workflow.wait_condition(lambda: self._reply_received or self._is_cancelled, timeout=timedelta(days=3))`.

2. Stage 2: First Follow-Up

- If no reply, checks kill switch and executes `send_single_email_activity` with Follow-Up 1 body.
- Suspends: `workflow.wait_condition(lambda: self._reply_received or self._is_cancelled, timeout=timedelta(days=7))`.

3. Stage 3: Second Follow-Up

- If still no reply, checks kill switch and sends Follow-Up 2.
 - Marks thread as `completed_no_reply` and exits.
-

5. OTHER PLATFORM WORKFLOWS

5.1 OnboardingWorkflow

Manages client onboarding and competitor intelligence extraction.

- **Trace:**

1. `validate_client_domain_activity` (verifies DNS resolution, TLS certs).
2. `enrich_business_profile_activity` (calls LLM to parse target business description and keyword tags).
3. `discover_competitors_activity` (calls Ahrefs/DataForSEO to identify top overlapping organic keywords).
4. `save_onboarding_results_activity` (writes DB record, sets client status to `active`).

5.2 KeywordResearchWorkflow

Performs automated keyword discovery and clustering.

- **Trace:**

1. `generate_seed_keywords_activity` (LLM-based seed expansion).
2. `expand_keywords_activity` (calls DataForSEO keyword research database).
3. `enrich_keywords_activity` (Ahrefs metric lookup for volume/difficulty).
4. `generate_keyword_embeddings_activity` (routes calls to nv-embedqa-e5-v5, writes vectors to Qdrant).
5. `cluster_keywords_activity` (runs HDBSCAN density clustering on vector values).
6. `name_clusters_activity` (LLM names categories based on token grouping).
7. **Approval Gate:** Human review of clusters.
8. `persist_keyword_data_activity` (writes cluster and keyword rows to DB).

5.3 CitationSubmissionWorkflow

Automates citation directories registration (Phase 2 & 3).

- **Trace:**

1. `validate_business_profile_activity` (ensures NAP consistency).
 2. `governance_scan_activity` (checks if listing already exists).
 3. `execute_directory_submission_activity` (browser-based automation filling directory submission forms).
 4. `create_citation_approval_activity` (notifies operator for verification).
 5. `verify_citation_listing_activity` (performs periodic HTTP lookup to ensure listing is active).
-

6. RETRY POLICIES SPECIFICATION REFERENCE

Retry settings are standardized inside `RetryPreset` as follows:

Preset Name	Initial Interval	Backoff Coefficient	Max Interval	Max Attempts	Non-Retryable Errors
<code>EXTERNAL_API</code>	<code>2s</code>	<code>2.0</code>	<code>300s</code>	<code>5</code>	<code>AhrefsRateLimitError</code> , <code>HunterAuthError</code>
<code>LLM_INFERENCE</code>	<code>3s</code>	<code>2.0</code>	<code>120s</code>	<code>3</code>	<code>NvidiaApiKeyInvalid</code> , <code>TokenLimitExceeded</code>
<code>DATABASE</code>	<code>1s</code>	<code>2.0</code>	<code>30s</code>	<code>5</code>	<code>DatabaseAuthError</code> , <code>DataIntegrityViolation</code>
<code>SCRAPING</code>	<code>5s</code>	<code>3.0</code>	<code>600s</code>	<code>3</code>	<code>RobotExclusionError</code> , <code>Http404Error</code>
<code>EMAIL_SEND</code>	<code>10s</code>	<code>2.0</code>	<code>300s</code>	<code>3</code>	<code>InvalidRecipientEmail</code> , <code>SpamRejectedError</code>
<code>TRANSIENT_IDEMPOTENT</code>	<code>2s</code>	<code>1.5</code>	<code>30s</code>	<code>10</code>	None

7. IDEMPOTENCY STRATEGY

To prevent duplicate actions during activity retries (e.g. sending duplicate emails, charging credits, running duplicate Ahrefs queries), all activities implement an idempotency store pattern backed by Redis.

```
# Standard Activity Pattern

async def send_single_email_activity(ctx, tenant_id, campaign_id, thread_id, to, subject, body):
    idempotency_key = f"email:{campaign_id}:{thread_id}:{to}"

    # Step 1: Check if key exists in Redis
    if await idempotency_store.exists(idempotency_key):
        logger.info("activity_already_executed", key=idempotency_key)
        return await idempotency_store.get(idempotency_key)

    # Step 2: Perform side effect
    result = await email_provider.send_email(...)

    # Step 3: Save results to Redis with 7-day TTL
    await idempotency_store.set(idempotency_key, result, ttl=3600*24*7)
    return result
```

This pattern ensures that if Temporal retries the activity because of a connection drop right after the email was sent, the retried execution will return the cached result and won't dispatch a second email.